STRATA: A Stratified Process-Mining Framework for HVAC Fault Detection and Diagnosis

Yassh Singh
Thompson Rivers University, Kamloops, Canada
Undergraduate Research Experience Award Program (U-REAP) 2026
Supervisors: Dr. Anthony Aighobahi and Dr. Mridula Sharma
singhy21@mytru.ca

Abstract—Faults and poor control in heating, ventilation and air-conditioning (HVAC) equipment waste 15 to 30 percent of a commercial building's energy, and many go unnoticed. Automated tools either apply rules written in advance, which miss any fault not anticipated, or report anomaly scores that say a day is unusual without saying where or what is wrong. This paper presents STRATA, a framework that works from the trend data a building already records and learns, from one period of fault-free operation, what each day should look like. It then examines every new day with eight checks, each asking whether a physical relationship or an operating sequence has moved outside its normal range. Each check is first run on fault-free days it has never seen, and the number it wrongly flags is its false-alarm rate; a check may raise an alarm only when it flags far more days than that rate could explain. Process mining supplies the structure: the sensor data becomes a log of events along the control sequence, and models of normal operation are discovered from that log for each zone's equipment and for the whole unit, which is how every alarm knows where it belongs. Every alarm states what broke, where, the evidence and the check's measured false-alarm rate. On five public benchmark systems from Lawrence Berkeley National Laboratory, each simulated with known seeded faults, STRATA detected 157 of 175 faults, most on the first day, and raised false alarms on 26 of 480 fault-free days it had never seen. In the 50 cases where the fault lay inside one zone's equipment, the alarm pointed to the correct zone in 37, to no zone in 13, and to a wrong zone in none. Moving to a new building means writing one configuration file, not new code. The next step is a live trial on a real building, and the step after that is ProcessHeal, a self-healing building that finds its own faults, locates them, works out their cause and corrects them.

Index Terms—HVAC, fault detection, process mining, building automation, event logs, explainable alarms

I. Introduction

Buildings account for about a third of energy-related carbon emissions [1], and HVAC accounts for up to half of a building's energy use [2]. Faults and poor control that go uncorrected waste 15 to 30 percent of a commercial building's energy [3]. A study of more than 60,000 pieces of HVAC equipment found 40 percent of air handlers in a faulted state on any given day, in buildings that already had automation systems with alarms [4]. Seven of the ten most common air-handler faults in that study are faults of behaviour, such as a wrong schedule or an unreleased manual override, rather than a broken part.

A building operator today usually learns of such a fault in one of three ways. An occupant complains that a room is too hot or too cold. The operator notices that a line on a trend graph has drifted from where it usually sits. Or an automated detector raises an alarm. The first two find the fault late; the third works only as well as the method behind it.

Those automated methods fall into two groups. Rule libraries, which come built into commercial building automation systems, are transparent and need no training data, but they catch only the faults for which someone wrote a rule. Machine-learning detectors learn from the building's own data and so can find faults that no rule anticipated, but most report an anomaly score: a number that says today is unusual without saying where or what is wrong. A score is not enough to write a work order.

This paper presents STRATA. It is configured with rules, written once per building in a single file, but those rules do not raise alarms. They turn the sensor data into a log of events: cooling started, damper open while commanded closed, a supply-air reading that physics says cannot be right. What a normal day of those events looks like is then learned from a period of the building's own fault-free operation, and every new day is checked against it, so a fault can show up as a departure from normal even when no rule names it. Each alarm states the fault, the piece of equipment, the evidence, and how often that same check has been wrong on fault-free days.

The method behind it is process mining, which discovers a model of a process from a log of events and checks later behaviour against the model [5]. STRATA builds such a log from the sensor data, for each zone's equipment and for the whole unit, which is how an alarm knows where it belongs. To our knowledge this is the first use of process mining for HVAC fault detection (Section II). On five public benchmark systems with 175 seeded faults, STRATA detected 157, most on the first day, and placed no alarm in a wrong zone (Section IV). The code and the complete evaluation are public.[1]

[1]https://github.com/iyassh/strata

II. Related Work

A. Rule-based detection, the method in use today

Fault detection in real buildings is almost entirely rule-based. The air-handling-unit performance assessment rules of Schein and colleagues set the pattern [6]. A rule states a relationship that must hold in a given operating mode, such as no water flow through a closed valve, and fires when it does not. ASHRAE Guideline 36 publishes such fault conditions alongside its control sequences [7], [8], and an open-source implementation of them exists [9]. A survey of fourteen commercial fault-detection tools found this rule-and-threshold style to be the norm [10].

Detection pays when it is used. Among 26 organisations running fault-detection software on 550 buildings, the median energy saving was 8 percent [11]. In a two-site field trial of Guideline 36, the facility team paid no attention to the built-in fault detection [12]. A 2026 California code study names alarm overload and a shortage of trained staff as the main barriers [13], and when alarms outnumber the people who can act on them, false alarms erode trust in the true ones [14].

Rules work from the first day and need no baseline data. Their limit is structural: they miss any fault that no rule author anticipated. The Lawrence Berkeley National Laboratory (LBNL) group itself notes that commercial diagnostics often list several candidate causes or none [15]. The same group has carried the rule approach through to correction, field-testing seven hand-coded correction algorithms on 225 pieces of equipment in four buildings [16], [17].

B. Machine learning, and what its alarms leave out

A 2024 systematic review covers 211 studies of machine learning for HVAC fault detection [2]. On the standard benchmarks, accuracy has saturated at 97 to 98 percent [18], [19]. What has not been solved is the output. Where an explanation is offered, it comes from a separate layer that ranks the sensors by how much each one swayed the verdict [20], [21], [22]. That answers which signal mattered, not what the equipment should have been doing. Two 2026 studies come closest to a located alarm. One estimates the position of a stuck damper or valve after training on fault-free data alone [23]. The other weighs the evidence for each component that could be at fault and names the most likely one, without labels [24].

The second problem is labels. Real buildings have none, so the strongest recent work trains only on normal operation and flags whatever departs from it [25], [26], and what it reports is a single deviation score. A model learned on one building also travels badly to another: in a 2026 study, detectors lost almost half their performance on buildings they had not seen [27], and two earlier studies saw the same [28], [29]. An audit of 65 such studies found two that shared their code [30].

Two recent hybrids use hand-written rules to label historical data and then train a classifier on those labels: one on a Danish university dataset [31], the other on a real air handler in Norway with the Schein rules as its rule layer [32]. The rules are still written in advance, so the classifier can learn only the faults the rules already name, and the output is still a class label without a location. A newer direction has a large language model write and refine the rules under physical constraints and report its evidence; it beats the neural detectors on the LBNL air-handler data [33], although a 2026 review of 66 such studies found none in sustained deployment [34].

C. Process mining, and how close it has come to buildings

The discovery algorithm used here, the inductive miner, is the standard one [35]. Conformance checking lines a day's events up against the model and shows where they depart from it [36], so the explanation comes with the method rather than from a layer added afterwards. Applying it to sensor data [37] needs one extra step, event abstraction, which turns continuous signals into discrete events [38], [39].

With that step the method has reached several physical systems. It detects anomalies in chemical plants [40], in industrial alarm sequences [41] and in industrial sensor streams [42]. One group has carried it to digital twins and to fault diagnosis on a robotic arm, and shows that features drawn from the process model make a detector far better than the plain check alone [43], [44], [45]. The closest analogue to a building is pasteurisation, where sensor signals are abstracted into activities and checked against the critical control points of the HACCP food-safety standard [46]; the parallel between those control points and Guideline 36 control sequences is the one STRATA builds on. Two kinds of work sound similar but are not this. Petri nets of HVAC equipment drawn by hand exist [47], but a net an engineer draws is not a model discovered from a log. And LBNL's OpenBuildingControl verifies that a control implementation conforms to its Guideline 36 specification before the building runs [48]; STRATA checks what the building does afterwards, from its operating data.

D. The gap, and where STRATA sits

No published work applies process mining to building HVAC fault detection. The 211-study review has no process-mining entry in its taxonomy of 23 methods [2], and the LBNL taxonomy of fault-detection methods does not list it [10]. From the other side, a review of 47 process-mining studies in Industry 4.0 [49], a review of 36 on sensor data covering industry, smart homes and healthcare [39], and a review of 93 on process management with the Internet of Things [50] contain no building application. Eight Google Scholar searches in February 2026 found no such paper, and a title-and-abstract search of the OpenAlex index for process mining together with HVAC, air handling, building automation, building energy or chillers, run on 16 October 2026, returns no work since

2023. The nearest work analyses building-management alarm logs statistically, without discovering a process model from them [51].

The gap is not hard to explain: the two communities do not meet, and buildings log continuous sensor streams rather than discrete events, so the event-abstraction step of Section II-C stands in the way.

Katipamula and Brambley divide the problem into detection, isolation and identification: noticing that a fault exists, locating it, and naming its cause [3]. Only the two 2026 studies above reach isolation. STRATA does detection and isolation, and sits between the two groups: it learns normal operation from fault-free data, which a rule library cannot, and reports a location, which a score cannot. It is a complement to a rule library rather than a replacement, and its located alarms are the kind of input the LBNL correction algorithms were built to act on. Identification of the cause, and correction, are the later stages of ProcessHeal, the self-healing building toward which STRATA is the first step.

III. The Framework, Stage by Stage

Figure 1 shows the framework as four tiers. The top two tiers are built once per building from one fault-free period; the bottom two run on every new day. The stages below follow the figure from top to bottom.

A. Stage 1: the building's data and one configuration file

The input is the trend data a building-automation system already records, usually once a minute: temperatures, flows, pressures, damper and valve positions, fan speeds. No new sensors are needed. The framework needs one fault-free period of that data, ideally a year so that every season is seen, and one configuration file. That file is the only thing specific to a building. It does two things. First, it maps the building's sensor names to the roles the framework understands, so that a point known in the building only by a code is known to the framework as, say, the supply-air temperature. Second, it writes down how the equipment is supposed to behave, as rules. The rules come from ASHRAE Guideline 36 [7], the industry's published standard for how HVAC equipment should be controlled, and from the equipment's own documents, which often state what its readings should be. A few more rules we wrote ourselves, later, after the models of Stage 3 showed us things that fault-free days always do. There are two kinds of rule. State rules describe what the unit is doing: whether it is occupied, whether it is cooling, whether it is bringing in outside air. Signature rules describe what must never happen, such as flow through a closed valve. In Stage 2 these two kinds of rule produce the two kinds of event shown in Figure 1. Onboarding a new building means writing this file, not changing code. On the benchmarks, configuring a second equipment type took about one and a half hours, and two of the five main systems were configured before any fault data was opened, so nothing in them could have been tuned to the faults.

B. Stage 2: turning sensor data into events

Process mining does not read sensor values. It reads a log of events, so the first thing the framework does with each day of data is turn it into one. The rules of Stage 1 do this. Every minute, the state rules are applied to the readings, and whenever the state of the equipment changes, an event is written with its time: "fan started at 06:02", "cooling became active at 09:14". These are state events. They say what the equipment is doing and pass no judgement on whether it is right. The signature rules are applied in the same way, and whenever one is broken an event is written: "water flowing through a closed valve at 09:40". These are signature events. They say that something happened which should not. Each day becomes one sequence of events. The two kinds of event are kept apart from here on. They do two different jobs. A signature event catches a fault that someone named in advance. The model catches a day that departs from normal in a way nobody named. State events go to Stage 3, where the model of a normal day is discovered from them. Signature events never do; they go straight to the Rules check of Stage 4, the dashed arrow in Figure 1. The reason is circularity. If the model were discovered from a log that contained signature events, it would learn that fault-free days have none, and on a fault day it would flag the signature event as a departure from normal. That would look like process mining finding a fault, but it would only be our own rule being counted a second time. We measured this on one system: letting signature events into the model raised the unit conformance check's apparent detection from 1.8 to 5.2 percent of fault days, and every one of the added days was the model re-finding a rule we had written ourselves.

C. Stage 3: discovering what normal looks like

This is the process-mining step. Its input is the state events of every fault-free day, from Stage 2. Its output is a model of a normal day. A model is a small map of the states the equipment passes through and the order it passes through them in. For an air handler it might read: the fan starts, the occupied period begins, cooling becomes active, and later in the day each of these ends. Nobody draws this map. A standard process-mining algorithm, the inductive miner [35], reads all the fault-free days and finds the one map that they all follow. Process mining calls this map a Petri net. The framework discovers models at two levels, and these are the two strata that give STRATA its name. The first level is the unit, meaning the main piece of equipment, such as an air handler. Its model is the normal daily cycle of that equipment. The second level is the terminal unit, meaning the box in each zone that the air handler serves. Each box has a damper and a reheat valve, and each box gets its own model of the normal

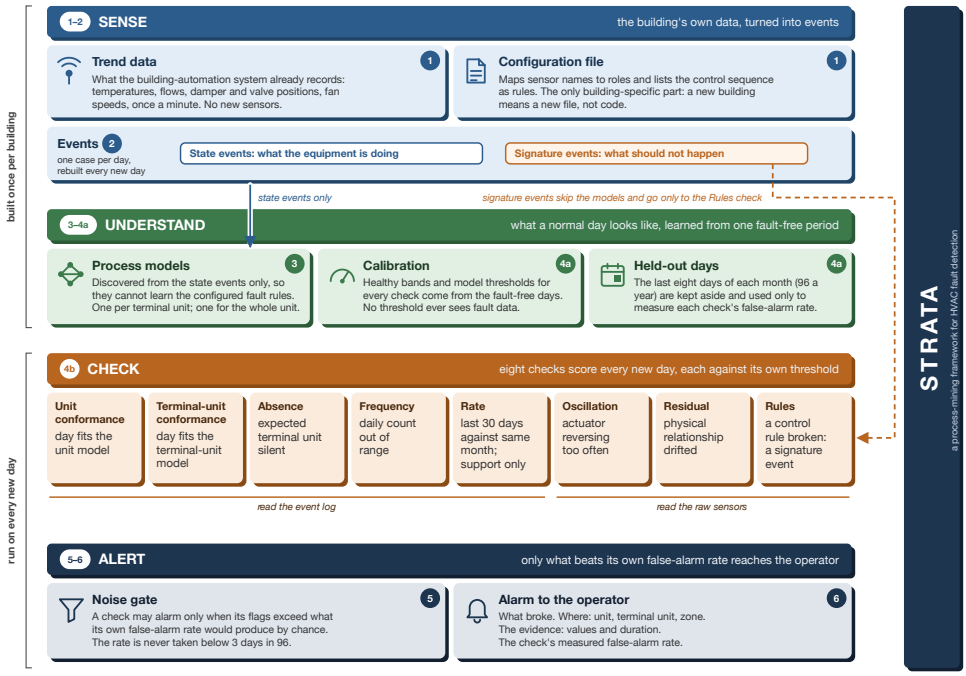


Fig. 1. The STRATA framework as four tiers. The top two are built once per building from one fault-free period; the bottom two run on every new day. The badges number the stages; 4a and 4b are the two halves of Stage 4. The arrows carry the rule that matters most: state events feed model discovery; signature events never do, and go only to the Rules check (Stage 2 says why).

day of those two parts. This is what gives an alarm its location. In Stage 4 a new day is compared against each model in turn. If the day fits the air handler's model and fits every terminal unit's model but one, the alarm names that one terminal unit. Reading the models during development also showed patterns of normal operation we had not thought to write down, such as the heating never running on an occupied workday; those became the extra rules of Stage 1.

D. Stage 4: calibration, then eight checks

Calibration (4a). Before any new day is scored, the framework learns from the fault-free period what normal looks like for this building, as numbers. Most checks work the same way. A check looks at one day and reduces it to one number, such as how many times the heating started. On fault-free days that number stays within some range, and that range is the check's healthy band; its edges are the check's thresholds. A day whose number falls outside the band is flagged. The last eight days of each month, 96 days in a year, are held out and used for one thing: counting how often each check flags a fault-free day, which is its false-alarm rate. The other days set the bands and train the models. The two conformance checks set their band on a further set of fault-free days, kept apart from both. No band ever sees fault data, and none sees the days on which its false-alarm rate is later reported. That separation, and the support-only status of the Rate check below, were decided after the first results showed the need.

The checks (4b). Every new day is scored, meaning that all eight checks are run on it, and an alarm names the check that raised it. The first three read the raw sensor values. The Rules check flags a day on which a signature event of Stage 2 occurred, such as flow through a closed valve. It is the one check that needs no band, because the rule itself says what is wrong. The Residual check uses a relationship that physics says must hold in a known state. With the cooling coil off, for example, the supply-air temperature should equal the mixed-air temperature plus the small rise the fan adds. The gap between the two sides is the day's number, and the day is flagged when that gap leaves the band. The Oscillation check counts how many times an actuator, such as a damper, reversed direction during the day. A count far above the fault-free band is the sign of an unstable control loop. The other five read the event log of Stage 2. The Unit conformance and Terminal-unit conformance checks take the day's state events and measure how well they fit the models of Stage 3, one against the unit's model and one against each terminal unit's model. A day fits poorly when, for example, cooling starts before the fan, an order the fault-free days never showed. The Absence check looks for a terminal unit that produces events on every fault-free occupied day and produced none on this occupied day. The Frequency check counts the events of each kind in the day, such as the number of heating episodes, and flags a count outside the band, whether too many or too few. The Rate check looks over a longer window. It counts on how many

```
STRATA alarm -- series fan-powered terminal
               units, 2018-01-01 (day 1 of fault)
 what       zone primary-air flow not tracking
          its setpoint          [check: rules]
 where       terminal unit TU_S, zone S
 evidence  715 CFM measured against a 200 CFM
          setpoint, a 515 CFM excess, held for
          720 min while occupied; damper 0.80
          open, 0.32 on the same date of the
          fault-free year
 rate       rules check: 0 false-alarm days on
          the 96 held-out fault-free days
```

Fig. 2. One alarm from the series fan-powered benchmark, every value taken from the framework's recorded output. The seeded fault was the zone-S damper stuck at 80 percent.

of the last 30 days a state, such as heating, was active, and compares that with the same month of the fault-free year. This check never raises an alarm on its own, because on one system it added false alarms. It is reported beside another check's alarm as supporting evidence.

E. Stage 5: the noise gate

A flagged day is not yet an alarm. Every check is sometimes wrong on a healthy day, and Stage 4 measured how often: that is its false-alarm rate. So a few flagged days prove nothing; the check would produce them anyway. The gate asks one question: has this check flagged more days than its false-alarm rate can explain? The arithmetic is a binomial test, the same calculation used for coin tosses. Suppose a check wrongly flagged three of the 96 held-out days. If it then flags five days of a 365-day year, that is within its usual error and the gate stays shut. If it flags forty, that is far beyond chance and the check may raise an alarm. The rate is never taken below three in 96, so a check that happened to make no mistakes on the held-out days is not treated as perfect; the alarm still shows the true count, which can be zero. And on the benchmarks the gate is applied once per check per scenario, a scenario being the data recorded with one seeded fault, and the day of the alarm is the first day that check flagged.

F. Stage 6: the alarm the operator receives

What survives the gate reaches the operator, the person who maintains the building's HVAC, as one alarm with four parts: what broke, where, the evidence, and how often this check has raised a false alarm before. Figure 2 shows one. The seeded fault was a zone damper stuck open, and the alarm came on the first day of it. The what and where lines name the symptom and the terminal unit. The evidence line gives the measured flow against its setpoint and the damper position against the same date of the fault-free year. The rate line is the count from Stage 5, zero here, so the operator can see how much to trust it.

IV. Results on Public Benchmark Data

We tested the framework on the public fault-detection datasets of the Lawrence Berkeley National Laboratory (LBNL) [52]. Each dataset covers one kind of equipment and holds a year of fault-free operation, recorded every minute, and then the equipment recorded again, once for each fault put in, such as a stuck damper, a biased sensor or a fouled coil. Of the six datasets in Table I, five are the main systems: a single-duct air handler, a dual-duct air handler, a fan coil unit, and two systems of fan-powered terminal units. These five have a full year of data, and unless we say otherwise, every number below is for them. The sixth is a rooftop unit.

TABLE I
The six LBNL datasets. False-alarm days are held-out fault-free days with an alarm. †A fourteenth alarm was attributed on audit to a dataset defect rather than to the fault and is not counted.

System	Scored	Detected	False-alarm days
Single-duct air handler	14	13†	1 of 96
Fan-powered terminal units (parallel)	30	26	8 of 96
Fan-powered terminal units (series)	29	25	9 of 96
Dual-duct air handler	55	50	3 of 96
Fan coil unit	47	43	5 of 96
Rooftop unit	24	20	0 of 28

A. Detection and false alarms

Across the five main systems the framework detected 157 of 175 seeded faults, with 26 false-alarm days in 480 held-out fault-free days. The median time from the start of a fault to its first alarm was one day. One further alarm, on the single-duct air handler, was traced on audit to a defect in the dataset rather than to the fault, and is not counted. The first configuration files left some sensors unmapped; a later audit mapped every one, and sixteen of the 157 detections came from those sensors. The 17 faults missed are all coil fouling. On these datasets the recorded sensor values stay inside every healthy band, so nothing that reads them can see it. On the one dataset that records the sensors inside the cooling circuit, the framework caught fouling at every strength it was seeded.

B. Where the alarm points

Catching a fault is only useful if the alarm sends the operator to the right place, so we took the 157 alarms counted above and checked where each one pointed. When the fault was in a zone, the alarm never pointed at the wrong zone. We could test the zone only on the two fan-powered systems, the only ones with zones: each air handler serves four terminal units, and the benchmark puts a fault into one of them. That gave us 50 alarms with a known right answer. In 37 the alarm named the right terminal unit. In 13 it named the air handler but no terminal unit. In none did it name a wrong one. These 50 were counted on the first run, before the sensor audit. The other three systems are a single piece of equipment, so for them, and for all 157 alarms together, we asked a simpler question: did the alarm blame the right part, such as the coil, the fan or a damper? It blamed the right kind

TABLE II
STRATA against the Guideline 36 rules on the three systems where both were run.

	STRATA		G36 rules	
System	Detected	False	Detected	False
Single-duct air handler	13 of 14	1 of 96	12 of 14	1 of 96
Fan-powered terminal units (parallel)	26 of 30	8 of 96	8 of 30	0 of 96
Fan-powered terminal units (series)	25 of 29	9 of 96	9 of 29	0 of 96

of part in 83 percent of alarms. It blamed the exact part in 69 percent.

C. Against two baselines

To see whether the framework is progress on what exists, we ran two existing methods on the same data and compared. The first is a standard statistical detector, principal component analysis, which learns what all the sensors look like together on normal days and flags a day that looks different. Both were run on the same 73 faults. The framework caught 64 and the detector 61, a difference too small to be statistically significant. The progress is in the alarm: the detector gives a score with no location, and the framework names the terminal unit. The second method is what buildings run today, the Guideline 36 fault rules, taken from an independent open-source implementation. Its alarms went through the same noise gate as ours, so the two are scored alike. Table II shows the counts. On the single-duct air handler the framework matched the rules: one more fault caught, with the same number of false alarms. On the two terminal-unit systems it caught three times as many faults as the rules, because it reads the sensors in each zone and the rules do not. These are the faults that today's rules miss, and the framework found 51 of the 59. The gain has a price: the rules raised no false alarms on those systems, and the framework raised eight and nine.

D. What process mining contributes

Because process mining had not been applied to HVAC fault detection before, nobody knew what it would contribute, so we measured it. We began with the hypothesis that the process models of Stage 3 would detect the faults by themselves. The two conformance checks are the only checks that use the models, and over 199 scenarios on the six systems of Table I they caught no fault on their own. That finding is why detection rests on eight calibrated checks rather than on the models. What process mining contributes is structure. The model per terminal unit is what gives every alarm its location. Reading the models showed us regularities of normal operation that became rules (Stage 3). And the event log the models need is what the frequency check reads; on the two fan-powered systems that check is one of only two that see the unstable-controller faults. Process mining helps to get detection but is not detecting on its own.

V. Future Work

The next step is to run STRATA on a real building. Thompson Rivers University (TRU) is the planned site, and the plan has four steps. First, collect a fault-free period of trend data. Second, write the configuration file: every recorded point mapped to a role, and the control sequence as rules. Third, confirm silence: run the signature rules over the fault-free period and fix any that fire, then calibrate the checks and measure their false-alarm rates. Fourth, operate: run Stages 2 to 6 on every new day.

The benchmarks are simulated, and a real building brings the faults they do not seed: seven of the ten most common air-handler faults are behavioural, such as a wrong schedule. The trial is scored on the benchmark measures: faults detected, false-alarm days, days to the first alarm, and whether the alarm named the right equipment. Where the real building scores differently, the gap is what simulated data left out.

The step after the trial is self-healing. STRATA is the first stage of ProcessHeal, a building that finds its own faults, locates them, works out their cause and corrects them. The next stage is cause, and the model of Stage 3 already records which step of the normal sequence the day departed from. The last stage is correction, in which the building releases the override or restores the schedule itself and the same checks confirm the next day is normal.

VI. Conclusion

STRATA set out to turn the trend data a building already records into alarms an operator can act on. On five public benchmark systems it detected 157 of 175 seeded faults, most on the first day. It raised false alarms on 26 of 480 fault-free days. In the 50 cases where the fault lay in one zone, it never named the wrong zone. It found 51 of the 59 faults that the rules buildings run today miss. Every number was measured on days the framework had never seen. This is the first use of process mining for HVAC fault detection, and the report has shown what the method adds. The models discovered from the building's own data give each alarm its location, and the event log behind them lets the checks catch faults that no rule anticipated. What the operator receives is one alarm that names the fault, the equipment, the evidence, and the check's own record of false alarms. Moving to a new building takes one configuration file, not new code. STRATA is the finding and locating stage of ProcessHeal, a building that repairs itself, and the live trial at TRU is where that work begins.

Acknowledgment

The author thanks Dr. Anthony Aighobahi and Dr. Mridula Sharma for their supervision, and U-REAP for its support.

References

[1] United Nations Environment Programme and Global Alliance for Buildings and Construction, "Global status report for buildings and construction 2024/2025: Not just another brick in the wall," United Nations Environment Programme, Paris, Tech. Rep., 2025, doi: 10.59117/20.500.11822/47214. [Online]. Available: https://globalabc.org/resources/publications/global-status-report-buildings-and-construction-20242025-not-just-another

[2] J. Bi, H. Wang, E. Yan, C. Wang, K. Yan, L. Jiang, and B. Yang, "AI in HVAC fault detection and diagnosis: A systematic review," Energy Reviews, vol. 3, no. 2, p. 100071, 2024, doi: 10.1016/j.enrev.2024.100071.

[3] S. Katipamula and M. R. Brambley, "Methods for fault detection, diagnostics, and prognostics for building systems—a review, part I," HVAC&R Research, vol. 11, no. 1, pp. 3–25, 2005, doi: 10.1080/10789669.2005.10391123.

[4] E. Crowe, Y. Chen, H. Reeve, D. Yuill, A. Ebrahimifakhar, Y. Chen, L. Troup, A. Smith, and J. Granderson, "Empirical analysis of the prevalence of HVAC faults in commercial buildings," Science and Technology for the Built Environment, vol. 29, no. 10, pp. 1027–1038, 2023, doi: 10.1080/23744731.2023.2263324.

[5] W. M. P. van der Aalst, Process Mining: Data Science in Action, 2nd ed. Springer, 2016, doi: 10.1007/978-3-662-49851-4.

[6] J. Schein, S. T. Bushby, N. S. Castro, and J. M. House, "A rule-based fault detection method for air handling units," Energy and Buildings, vol. 38, no. 12, pp. 1485–1492, 2006, doi: 10.1016/j.enbuild.2006.04.014.

[7] ASHRAE, "ASHRAE Guideline 36-2021: High-performance sequences of operation for HVAC systems," American Society of Heating, Refrigerating and Air-Conditioning Engineers, Peachtree Corners, GA, 2021.

[8] ——, "ASHRAE Guideline 36-2024: High-performance sequences of operation for HVAC systems," American Society of Heating, Refrigerating and Air-Conditioning Engineers, Peachtree Corners, GA, 2024.

[9] B. Bartling et al., "open-fdd: open-source fault detection and diagnostics for HVAC implementing the ASHRAE Guideline 36 fault conditions," 2026, version 4.4.1, released 15 Aug. 2026; source at https://github.com/bbartling/open-fdd. [Online]. Available: https://pypi.org/project/open-fdd/

[10] J. Granderson, R. Singla, E. Mayhorn, P. Ehrlich, D. Vrabie, and S. Frank, "Characterization and survey of automated fault detection and diagnostic tools," Lawrence Berkeley National Laboratory, Berkeley, CA, Tech. Rep. LBNL-2001075, Nov. 2017. [Online]. Available: https://eta-publications.lbl.gov/sites/default/files/lbnl-2001075.pdf

[11] G. Lin, H. Kramer, and J. Granderson, "Building fault detection and diagnostics: Achieved savings, and methods to evaluate algorithm performance," Building and Environment, vol. 168, p. 106505, 2020, doi: 10.1016/j.buildenv.2019.106505.

[12] X. Zhou, C. Battisti, R. Singla, G. LaPalme, G. Paliaga, and B. Eubanks, "Field demonstration of ASHRAE Guideline 36 high-performance sequences of operation for HVAC systems," Slipstream, TRC, and University of Minnesota, for the Minnesota Department of Commerce, Conservation Applied Research and Development program, Final Report, Contract 187281, Dec. 2025. [Online]. Available: https://mn.gov/commerce-stat/energy/cards-2/187281/187281_slipstream_ashrae_guideline36_final_report_ada.pdf

[13] C. Battisti, Y. Li, A. Faramarzi, R. Singla, M. Mutmansky, M. Greene, and T. Jayarathne, "2028 California Energy Code CASE report: Nonresidential HVAC fault detection and diagnostics," California Statewide Codes and Standards Enhancement (CASE) Program, Final CASE Report, Jun. 2026. [Online]. Available: https://title24stakeholders.com/wp-content/uploads/2026/06/2028_CASE-Report_Nonresidential-HVAC-Fault-Detection-and-Diagnostics_MaCS.pdf

[14] A. R. Diallo, L. Homri, T. Boeuf, J.-Y. Dantan, and F. Bonnet, "Quantifying and mitigating alarm fatigue caused by fault detection systems," Reliability Engineering & System Safety, vol. 267, p. 111890, 2026, doi: 10.1016/j.ress.2025.111890.

[15] G. Lin, J. House, Y. Chen, J. Granderson, and W. Zhang, "Active multi-mode data analysis to improve fault diagnosis in AHUs," Energy and Buildings, vol. 337, p. 115621, 2025, doi: 10.1016/j.enbuild.2025.115621.

[16] G. Lin, M. Pritoni, Y. Chen, and J. Granderson, "Development and implementation of fault-correction algorithms in fault detection and diagnostics tools," Energies, vol. 13, no. 10, p. 2598, 2020, doi: 10.3390/en13102598.

[17] M. Pritoni, G. Lin, Y. Chen, R. Vitti, C. Weyandt, and J. Granderson, "From fault-detection to automated fault correction: A field study," Building and Environment, vol. 214, p. 108900, 2022, doi: 10.1016/j.buildenv.2022.108900.

[18] W. Tun, J. K.-W. Wong, and S.-H. Ling, "Hybrid random forest and support vector machine modeling for HVAC fault detection and diagnosis," Sensors, vol. 21, no. 24, p. 8163, 2021, doi: 10.3390/s21248163.

[19] Prince, B. Yoon, and P. Kumar, "Fault detection and diagnosis in air-handling unit (AHU) using improved hybrid 1D convolutional neural network," Systems, vol. 13, no. 5, p. 330, 2025, doi: 10.3390/systems13050330.

[20] Q. Deng, Z. Chen, W. Zhu, Z. Li, Y. Yuan, and Y. Wang, "Adaptive fusion graph convolutional network based interpretable fault diagnosis method for HVAC systems enhanced by unlabeled data," Energy and Buildings, vol. 324, p. 114901, 2024, doi: 10.1016/j.enbuild.2024.114901.

[21] G. Li, Z. Yao, L. Chen, T. Li, and C. Xu, "An interpretable graph convolutional neural network based fault diagnosis method for building energy systems," Building Simulation, vol. 17, no. 7, pp. 1113–1136, 2024, doi: 10.1007/s12273-024-1125-6.

[22] H. Wang, W. Yan, Y. Zhang, J. Yu, W. Gao, B. Zhou, and Y. Liu, "xMHAC-PDCNN: An explainable deep network with spatial feature embedding for fault detection and diagnosis of chillers under variable operating conditions," Building and Environment, vol. 284, p. 113504, 2025, doi: 10.1016/j.buildenv.2025.113504.

[23] K.-S. Park, J. H. Kim, and C. S. Park, "Explainable fault detection and diagnosis in HVAC systems using a Koopman operator with normal data," Energy and Buildings, vol. 350, p. 116580, 2026, doi: 10.1016/j.enbuild.2025.116580.

[24] M. Paolini, M. S. Piscitelli, and A. Capozzoli, "A label-free hybrid fault detection and diagnosis approach for HVAC systems using Bayesian networks," Energy and Buildings, vol. 351, p. 116658, 2026, doi: 10.1016/j.enbuild.2025.116658.

[25] M. A. F. Abdollah, R. Scoccia, and M. Aprile, "Transformer encoder based self-supervised learning for HVAC fault detection with unlabeled data," Building and Environment, vol. 258, p. 111568, 2024, doi: 10.1016/j.buildenv.2024.111568.

[26] K. El Mokhtari and J. J. McArthur, "Autoencoder-based fault detection using building automation system data," Advanced Engineering Informatics, vol. 62, p. 102810, 2024, doi: 10.1016/j.aei.2024.102810.

[27] S. Jiang, "Leakage-safe cross-domain AHU fault detection for building operation: adaptation reliability under limited target information," Energy and Buildings, vol. 372, p. 118275, 2026, doi: 10.1016/j.enbuild.2026.118275.

[28] X. Zhu, K. Chen, B. Anduv, X. Jin, and Z. Du, "Transfer learning based methodology for migration and application of fault detection and diagnosis between building chillers for improving energy efficiency," Building and Environment, vol. 200, p. 107957, 2021, doi: 10.1016/j.buildenv.2021.107957.

[29] N. Ghalamsiah, J. Wen, K. S. Candan, T. Wu, Z. O'Neill, and A. Aghaei, "Unsupervised domain adaptation for HVAC fault diagnosis using contrastive adaptation network," Energy and Buildings, vol. 337, p. 115659, 2025, doi: 10.1016/j.enbuild.2025.115659.

[30] A. Mukhtar, M. Hadwiger, F. Wotawa, and G. Schweiger, "Reproducibility of machine learning-based fault detection and diagnosis for HVAC systems in buildings: An empirical study," Energy and AI, vol. 22, p. 100658, 2025, doi: 10.1016/j.egyai.2025.100658.

[31] S. Mazzetto, "Hybrid predictive maintenance for building systems: Integrating rule-based and machine learning models for

fault detection using a high-resolution Danish dataset," Buildings, vol. 15, no. 4, p. 630, 2025, doi: 10.3390/buildings15040630.

[32] H. R. Y. Zabadi, F. A. Zemedkun, H. Hosamo, and D. Kraniotis, "Integration of digital twins and machine learning for predictive maintenance using APAR method rules in non-residential buildings," Frontiers in Built Environment, vol. 11, p. 1734945, 2026, doi: 10.3389/fbuil.2025.1734945.

[33] S. Lin and C. Hua, "Physics-informed large language models for HVAC anomaly detection with autonomous rule generation," NeurIPS 2025 Workshop on UrbanAI (oral); arXiv:2510.17146, 2025. [Online]. Available: https://arxiv.org/abs/2510.17146

[34] A. Neubauer, T. Hong, H. Li, M. Yu, A. Darbandi, Y. Fürst, and M. Kriegel, "Large language models for HVAC operations in building energy systems: A critical review of methods, applications, and deployment readiness," arXiv preprint arXiv:2609.05314, 2026. [Online]. Available: https://arxiv.org/abs/2609.05314

[35] S. J. J. Leemans, D. Fahland, and W. M. P. van der Aalst, "Discovering block-structured process models from event logs – a constructive approach," in Application and Theory of Petri Nets and Concurrency (PETRI NETS 2013), ser. Lecture Notes in Computer Science, vol. 7927. Springer, 2013, pp. 311–329, doi: 10.1007/978-3-642-38697-8_17.

[36] J. Carmona, B. van Dongen, A. Solti, and M. Weidlich, Conformance Checking: Relating Processes and Models. Springer, 2018, doi: 10.1007/978-3-319-99414-7.

[37] F. Bezerra and J. Wainer, "Algorithms for anomaly detection of traces in logs of process aware information systems," Information Systems, vol. 38, no. 1, pp. 33–44, 2013, doi: 10.1016/j.is.2012.04.004.

[38] D. Janssen, F. Mannhardt, A. Koschmider, and S. J. van Zelst, "Process model discovery from sensor event data," in Process Mining Workshops (ICPM 2020), ser. Lecture Notes in Business Information Processing, vol. 406. Springer, 2021, pp. 69–81, doi: 10.1007/978-3-030-72693-5_6.

[39] E. Brzychczy, M. Aleknonytė-Resch, D. Janssen, and A. Koschmider, "Process mining on sensor data: a review of related works," Knowledge and Information Systems, vol. 67, no. 6, pp. 4915–4948, 2025, doi: 10.1007/s10115-024-02297-y.

[40] Y. Shi, N. Zhang, X. Song, H. Li, and Q. Zhu, "Novel approach for industrial process anomaly detection based on process mining," Journal of Process Control, vol. 136, p. 103165, 2024, doi: 10.1016/j.jprocont.2024.103165.

[41] A. Norouzifar, A. Neshastegaran, and I. Izadi, "Modeling alarm sequences using process mining techniques," in 2026 34th International Conference on Electrical Engineering (ICEE), 2026, pp. 1428–1433, doi: 10.1109/icee70628.2026.11710567.

[42] E. J. Husom, A. Goknil, F. Mannhardt, S. Tverdal, S. Sen, and P. Nguyen, "Unsupervised learning and process analysis for sensor data validation in the IIoT," ACM Transactions on Internet Technology, vol. 26, no. 2, pp. 1–34, 2026, doi: 10.1145/3777897.

[43] F. Vitale, S. Guarino, F. Flammini, L. Faramondi, N. Mazzocca, and R. Setola, "Process mining for digital twin development of industrial cyber-physical systems," IEEE Transactions on Industrial Informatics, vol. 21, no. 1, pp. 866–875, 2025, doi: 10.1109/TII.2024.3465600.

[44] F. Vitale, N. Dall'Ora, S. Gaiardelli, E. Fraccaroli, N. Mazzocca, and F. Fummi, "Process mining-driven modeling and simulation to enhance fault diagnosis in cyber–physical systems," Journal of Manufacturing Systems, vol. 84, pp. 189–206, 2026, doi: 10.1016/j.jmsy.2025.12.005.

[45] F. Vitale, M. Pegoraro, W. M. P. van der Aalst, and N. Mazzocca, "Control-flow anomaly detection by process mining-based feature extraction and dimensionality reduction," Knowledge-Based Systems, vol. 310, p. 112970, 2025, doi: 10.1016/j.knosys.2025.112970.

[46] A. Moradbeikie, A. P. Ayub da Costa Barbon, I. M. Grigore, D. F. Barbin, and S. Barbon Junior, "Process mining of sensor data for predictive process monitoring: A HACCP-guided pasteurization study case," Systems, vol. 13, no. 11, p. 935, 2025, doi: 10.3390/systems13110935.

[47] W. Deabes, K. E. Bouazza, and W. Algthami, "Smart fuzzy Petri net-based temperature control framework for reducing building energy consumption," Sensors, vol. 23, no. 13, p. 5985, 2023, doi: 10.3390/s23135985.

[48] M. Wetter, P. Ehrlich, A. Gautier, M. Grahovac, P. Haves, J. Hu, A. Prakash, D. Robin, and K. Zhang, "OpenBuildingControl: Digitizing the control delivery from building energy modeling to specification, implementation and formal verification," Energy, vol. 238, p. 121501, 2022, doi: 10.1016/j.energy.2021.121501.

[49] K. Akhramovich, E. Serral, and C. Cetina, "A systematic literature review on the application of process mining to Industry 4.0," Knowledge and Information Systems, vol. 66, no. 5, pp. 2699–2746, 2024, doi: 10.1007/s10115-023-02042-x.

[50] F. De Luzi, F. Leotta, A. Marrella, and M. Mecella, "On the interplay between business process management and Internet-of-Things: A systematic literature review," Business & Information Systems Engineering, vol. 67, no. 2, pp. 265–288, 2025, doi: 10.1007/s12599-024-00859-6.

[51] Z. Khais Shahid, S. Saguna, and C. Åhlund, "Unsupervised analysis of EcoStruxure Building Operation (EBO) alarm logs in school buildings," in Proceedings of the 13th ACM International Conference on Systems for Energy-Efficient Buildings, Cities, and Transportation (BuildSys '26), 2026, pp. 282–292, doi: 10.1145/3744256.3812592.

[52] J. Granderson, G. Lin, Y. Chen, A. Casillas, J. Wen, Z. Chen, P. Im, S. Huang, and J. Ling, "A labeled dataset for building HVAC systems operating in faulted and fault-free states," Scientific Data, vol. 10, p. 342, 2023, doi: 10.1038/s41597-023-02197-w.